

CSN364- Container Technologies

Module-4

Topic 1- Docker Compose

By

Dr. Garima Verma

Docker Compose

Docker Compose is a tool used to **define and run multi-container Docker applications** using a single YAML file.

If you have multiple container – such as cont1, cont2.. Etc. then to start you need to use multiple runs.

```
docker run cont1
```

```
docker run cont2
```

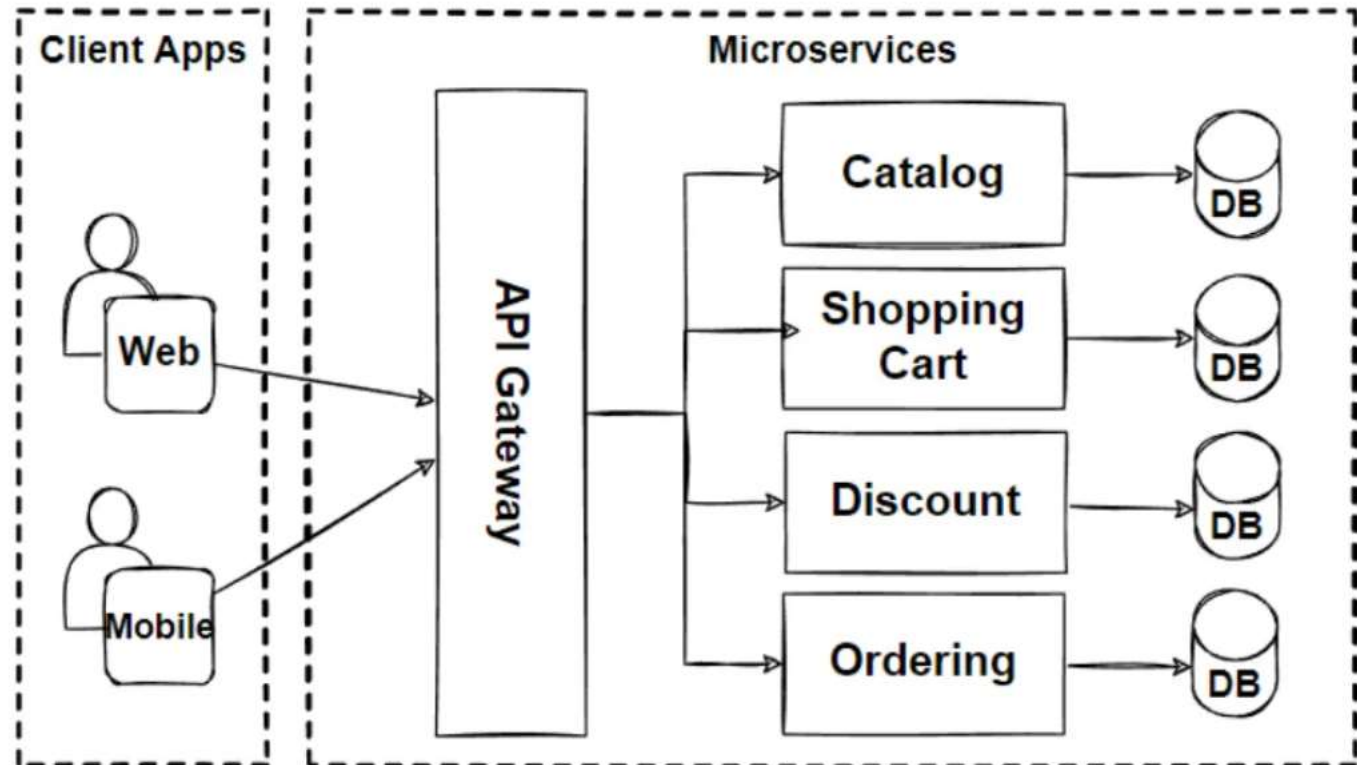
```
docker run ...
```

Why Docker Compose?

Modern applications are not single containers.

A typical web app needs:

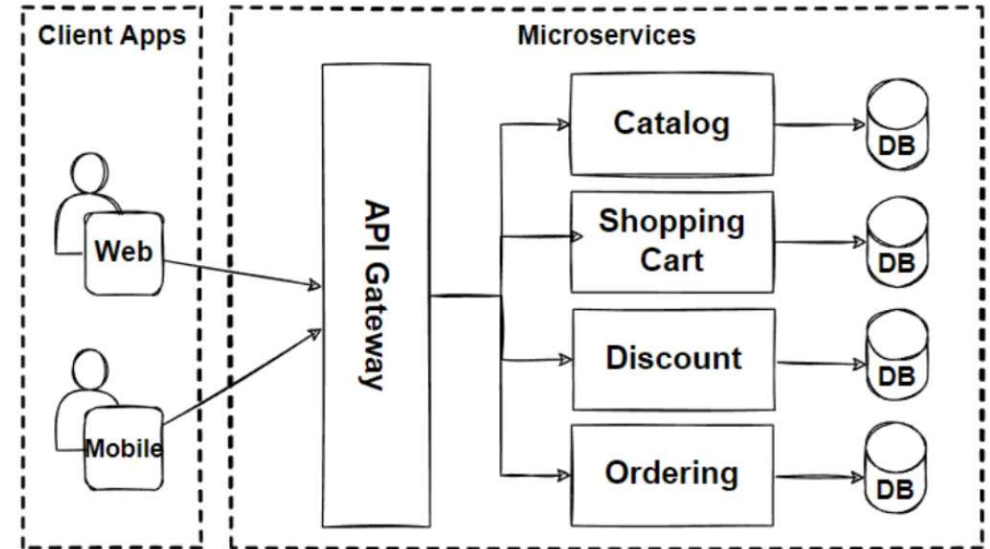
- Web server
- Application backend
- Database
- Cache / message broker



Why Docker Compose?

Docker Compose helps you:

- Run all services together
- Define networking automatically
- Manage volumes (persistent data)
- Start/stop everything with one command



Core Concepts

- **Services**

- Each container = **service**

Example:

web → login webpage

db → database

- **Networks**

- Compose creates a default private network
- Services can talk to each other using service names

Example: db:3306

Core Concepts

- **Volumes**
 - Used to **persist data**
 - Especially important for databases
- **docker-compose.yml Structure**

```
yaml

version: "3.9"

services:
  service_name:
    image:
    build:
    ports:
    environment:
    volumes:
```

Example: Login Webpage with Database

App structure-

Browser



Login Webpage (Flask)



MySQL Database

login-app/

|

|— docker-compose.yml

|— app/

| |— Dockerfile

| |— app.py

| └─ requirements.txt

Example: Login Webpage with Database

Step-1 Create Backend application (Flask) – app.py

```
from flask import Flask, request, render_template_string
import mysql.connector
import time
app = Flask(__name__)

# --- Retry until MySQL is ready ---
while True:
    try:
        db = mysql.connector.connect(
            host="db",
            user="root",
            password="1234",
            database="logindb"
        )
        print("-> Connected to MySQL database")
        break
    except mysql.connector.Error as err:
        print("⌚ Waiting for MySQL to be ready...", err)
        time.sleep(3)
```

Example: Login Webpage with Database

--- Routes ---

```
@app.route("/", methods=["GET", "POST"])
```

```
def login():
```

```
    msg = ""
```

```
    if request.method == "POST":
```

```
        user = request.form["username"]
```

```
        pwd = request.form["password"]
```

```
        cursor = db.cursor()
```

```
        cursor.execute(
```

```
            "SELECT * FROM users WHERE username=%s AND password=%s",
```

```
            (user, pwd)
```

```
        )
```

```
        result = cursor.fetchone()
```

```
        cursor.close()
```

Example: Login Webpage with Database

if result:

msg = "Login successful!"

else:

msg = "Invalid credentials"

return render_template_string("""

<h2>Login Page</h2>

<form method="post">

Username: <input name="username">

Password: <input name="password" type="password">

<button type="submit">Login</button>

</form>

<p>{{ msg }}</p>

""", msg=msg)

--- Run app ---

app.run(host="0.0.0.0", port=5000)

Example: Login Webpage with Database

Step-2 Create requirements.txt

```
app >  requirements.txt
1 flask
2 mysql-connector-python
```

Step-3 Create Dockerfile

```
app >  Dockerfile
1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install -r requirements.txt
7
8 COPY app.py .
9
10 CMD ["python", "app.py"]
```

Example: Login Webpage with Database

Step-2 Create requirements.txt

```
app >  requirements.txt
1 flask
2 mysql-connector-python
```

Step-3 Create Dockerfile

```
app >  Dockerfile
1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install -r requirements.txt
7
8 COPY app.py .
9
10 CMD ["python", "app.py"]
```

Example: Login Webpage with Database

Step-4 Create docker-compose.yml

```
docker-compose.yml
1  version: "3.9"
2
3  services:
4    web:
5      build: ./app
6      ports:
7        - "5000:5000"
8      depends_on:
9        - db
10
11   db:
12     image: mysql:8.0
13     environment:
14       MYSQL_ROOT_PASSWORD: 1234
15       MYSQL_DATABASE: logindb
16     volumes:
17       - db_data:/var/lib/mysql
18
19  volumes:
20    db_data:
```

Example: Login Webpage with Database

Step-5 Run command

docker compose up --build

Step-6 Check running containers—

docker ps

Step-7 goto terminal and run mysql container

docker exec -it dockercompose-db-1 mysql -u root -p

Step-8 Enter password – 1234

Step-9 Now use following commands in mysql

Example: Login Webpage with Database

- **Change database –**

USE logindb;

- **Create table users –**

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50),  
  password VARCHAR(50)  
);
```

- **Insert one user**

```
INSERT INTO users (username, password)  
VALUES ('admin', 'admin123');
```

- **Exit from mysql –**

Exit ;

Example: Login Webpage with Database

- Open browser

<http://localhost:5000>

User name – admin

Password – admin123

Login Page

Username:
Password:

Login successful!

REMEMBER – If anything goes wrong while creating application then use following command , correct code the again compose up.

`docker compose down -v`

Scale web

`docker compose up --build --scale web=3`

1 MySQL container → single database (stateful)

3 Flask containers → your web app (stateless)

All 3 Flask containers connect to the same MySQL container

Docker Compose Vs. Docker Swarm

Feature	Docker Compose	Docker Swarm
Scope	Single machine	Cluster (multi-node)
Main use	Dev / test	Production
Config file	docker-compose.yml	docker-compose.yml (with Swarm extensions)
Scaling	--scale (local)	replicas + auto scheduling
Load balancing	✗ No	✓ Built-in
High availability	✗ No	✓ Yes
Self-healing	✗ No	✓ Yes
Complexity	Very low	Medium